



Web Dev III – Lab 1 Viva & Assignment Guide

Assignment: Smart Utility Toolkit (Node.js Core Modules)

Course: Web Dev III (Node.js & Express Backend) | **Unit:** Unit-1

Marks: 2.5

Zero External Packages

100% Core Node.js



1. Project Implementation Summary

Task / Feature	File Location	Core Concept	Status
CLI Calculator	Lecture 1/calculator.js	Command line arguments (<code>process.argv</code>)	Completed
Custom Modules	Lecture 1/modules/isEven.js	CommonJS (<code>module.exports</code> & <code>require</code>)	Completed
Timestamp Logger	Lecture 1/modules/logger.js	ANSI colors & timestamp formatting	Completed
HTTP Server	Lecture 1/server.js	Built-in http module & custom routes	Completed
File Manager CRUD	Lecture 1/fileManager.js	Async fs (<code>writeFile</code> , <code>readFile</code> , <code>append</code> , <code>unlink</code>)	Completed
Random Dice Simulator	Lecture 1/dice.js	Cryptographic randomness (<code>crypto.randomInt</code>)	Completed
Master Launcher	Lecture 1/index.js	Unified command dispatcher	Completed



2. Core Node.js Fundamentals (Viva Q&A)

Q1. What is Node.js and why do we use it on the backend?

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's **V8 JavaScript engine**. It allows JavaScript to execute outside the browser to build fast, scalable, and event-driven network applications.

Q2. What are Built-in (Core) Modules in Node.js?

Core modules come pre-bundled with Node.js binary without requiring any `npm install`. In this lab, we utilized four core modules:

- **process** : Interacts with runtime environment and CLI arguments.
- **http** : Creates HTTP servers and handles web traffic.
- **fs** : Performs File System CRUD operations.
- **crypto** : Handles cryptographic security and true random number generation.

Q3. What is the difference between Synchronous and Asynchronous operations in Node.js?

- **Synchronous (Blocking):** Blocks the single thread until completion (e.g. `fs.readFileSync`). Slows down concurrent users.
- **Asynchronous (Non-Blocking):** Executes in background and triggers a callback/promise upon completion (e.g. `fs.readFile`). Enables high concurrency.

3. File-by-File Technical Viva Questions

Q4. How does the CLI Calculator work with `process.argv`?

`process.argv` is an array containing command-line arguments:

- `process.argv[0]` = Node executable path.
- `process.argv[1]` = JavaScript file path being run.
- `process.argv.slice(2)` = Actual user arguments (e.g. `['add', '10', '5']`).
- All inputs from `process.argv` are strings, so we convert them using `parseFloat()` before calculation.

Q5. Explain CommonJS module exports and require (`isEven.js` & `app.js`):

- **Exporting:** In `modules/isEven.js`, we use `module.exports = isEven;` to make the function available externally.
- **Importing:** In `app.js`, we use `const isEven = require('./modules/isEven');`.
- The `./` is mandatory to specify a relative path for local custom modules rather than a built-in module.

Q6. How does the HTTP Server work without Express (`server.js`)?

We use `http.createServer((req, res) => { ... })`:

- `req.url` is inspected to route requests to `/`, `/about`, `/contact`, or 404 handler.
- `res.writeHead(200, { 'Content-Type': 'text/plain' })` sets status code and headers.
- `res.end(data)` delivers the body and closes the HTTP stream.

Q7. Explain the 4 CRUD operations in `fileManager.js`:

We use asynchronous callback methods from the `fs` module:

1. **Create:** `fs.writeFile('sample.txt', 'Hello Node.js', callback)`
2. **Read:** `fs.readFile('sample.txt', 'utf8', callback)`
3. **Update:** `fs.appendFile('sample.txt', 'Learning FS Module', callback)`
4. **Delete:** `fs.unlink('sample.txt', callback)`

Q8. Why use `crypto.randomInt()` in `dice.js` instead of `Math.random()`?

`Math.random()` produces pseudo-random numbers that are predictable. The `crypto` module uses hardware entropy / OS-level random pool to produce cryptographically secure, uniformly distributed integers (e.g. `crypto.randomInt(1, 7)` for dice values 1 to 6).

★ Professor Viva Tips (How to Answer in Class)

- **If asked to run:** Use terminal command `node calculator.js add 10 5` or `node index.js`.
- **Mention Key Words:** "Non-blocking event loop", "CommonJS module system", "Built-in core modules", "Cryptographic entropy".
- **Why zero dependencies:** Emphasize that unit-1 focuses on mastering foundational Node.js internals before introducing Express.js.

